Spatiotemporal modeling workflows with *sdmTMB*: Hair crab index standardization

Sean Hardison, PhD

Rivanna Statistical Ecology Group LLC

In this lecture

Abundance index standardization analysis for hair crab (*Erimacrus isenbeckii*) collected by the NMFS Eastern Bering Sea bottom trawl survey

Along the way we will discuss:

- Preliminary data processing choices and analyses
- Best practices for creating meshes
- Fitting *sdmTMB* models and parsing outputs
- Model and residual diagnostics and fixing problems
- Generating prediction grids
- Calculating abundance indices
- *sdmTMB* model comparison and selection
- Bayesian inference

Follow along in R

1. We will briefly visualize and discuss hair crab CPUE data in `R/data_processing_and_visualization.qmd`

2. The index standardization analysis is in `R/sdmTMB_index_standardization.qmd`

3. We will do model comparison in `R/sdmTMB_model_comparison.qmd`

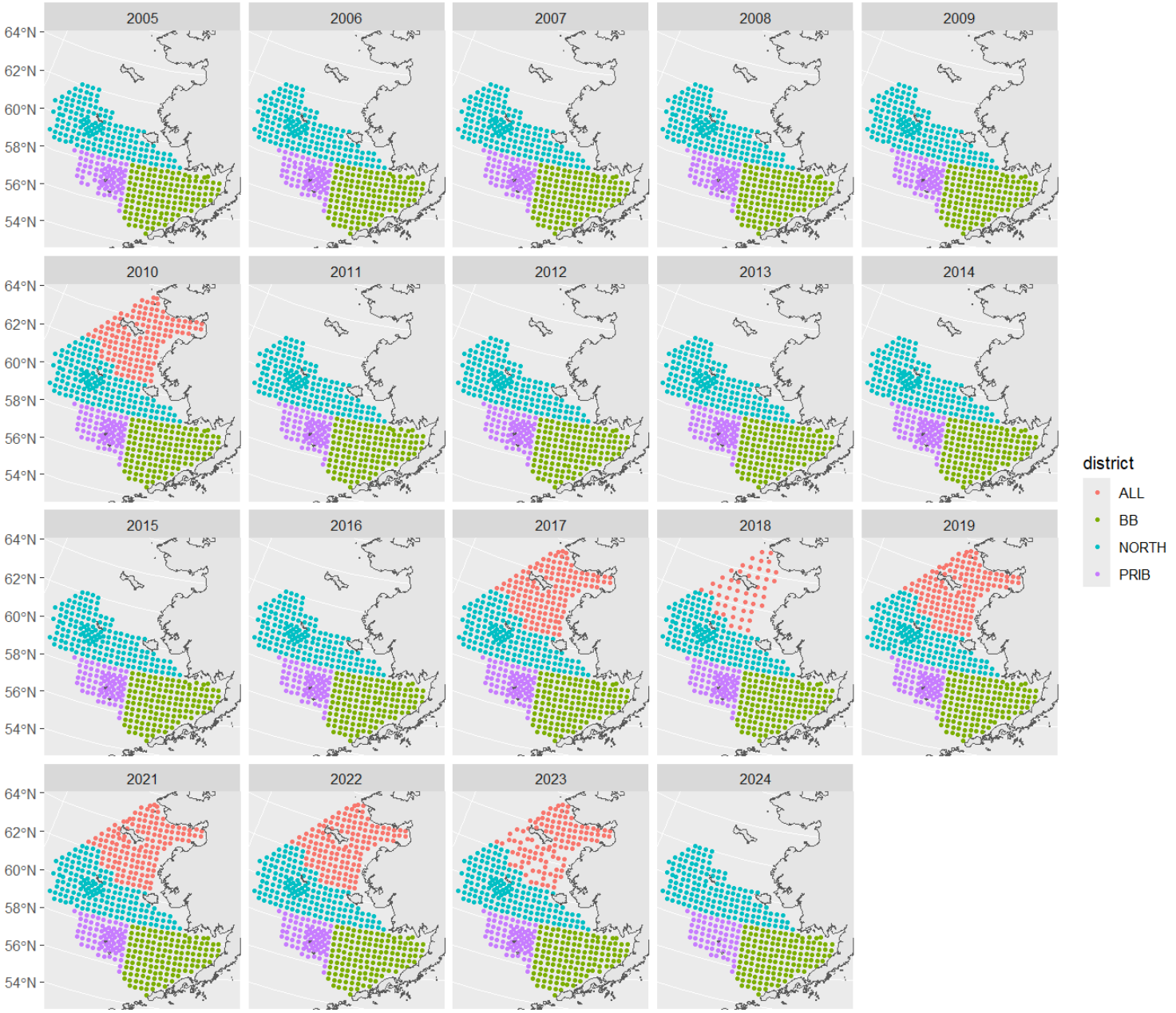
Preliminary data explorations

Survey effort distributions of NMFS Eastern and Northern Bering Sea bottom trawl surveys 2005-2024

Northern Bering Sea survey (red) does not occur every year

- Northern Bering Sea sampling effort not spatially uniform in all years

Higher density sampling effort ("corner stations") near Pribilof and St. Matthews Islands



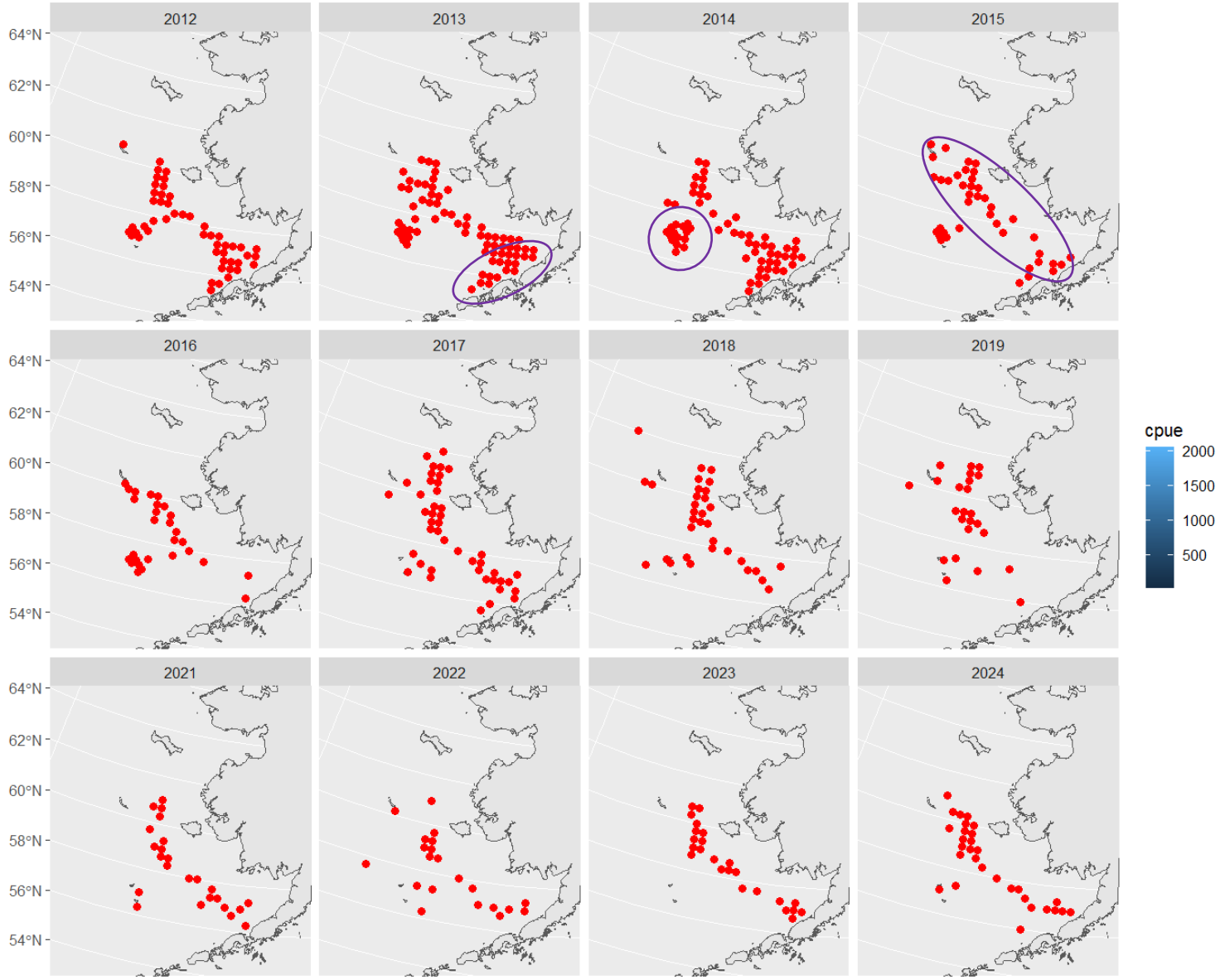
Preliminary data explorations

Index standardization will target legal male hair crab (>83 mm carapace length [CL])

Map is positive CPUE 2012-2024

Note clustered spatial distribution:

- Positive catches in NE-SW direction along Alaska Peninsula
- Bristol Bay population extends NW to southern Northern Bering Sea
- Population associated with Pribilof Islands



R/data_processing_and_visualization.qmd

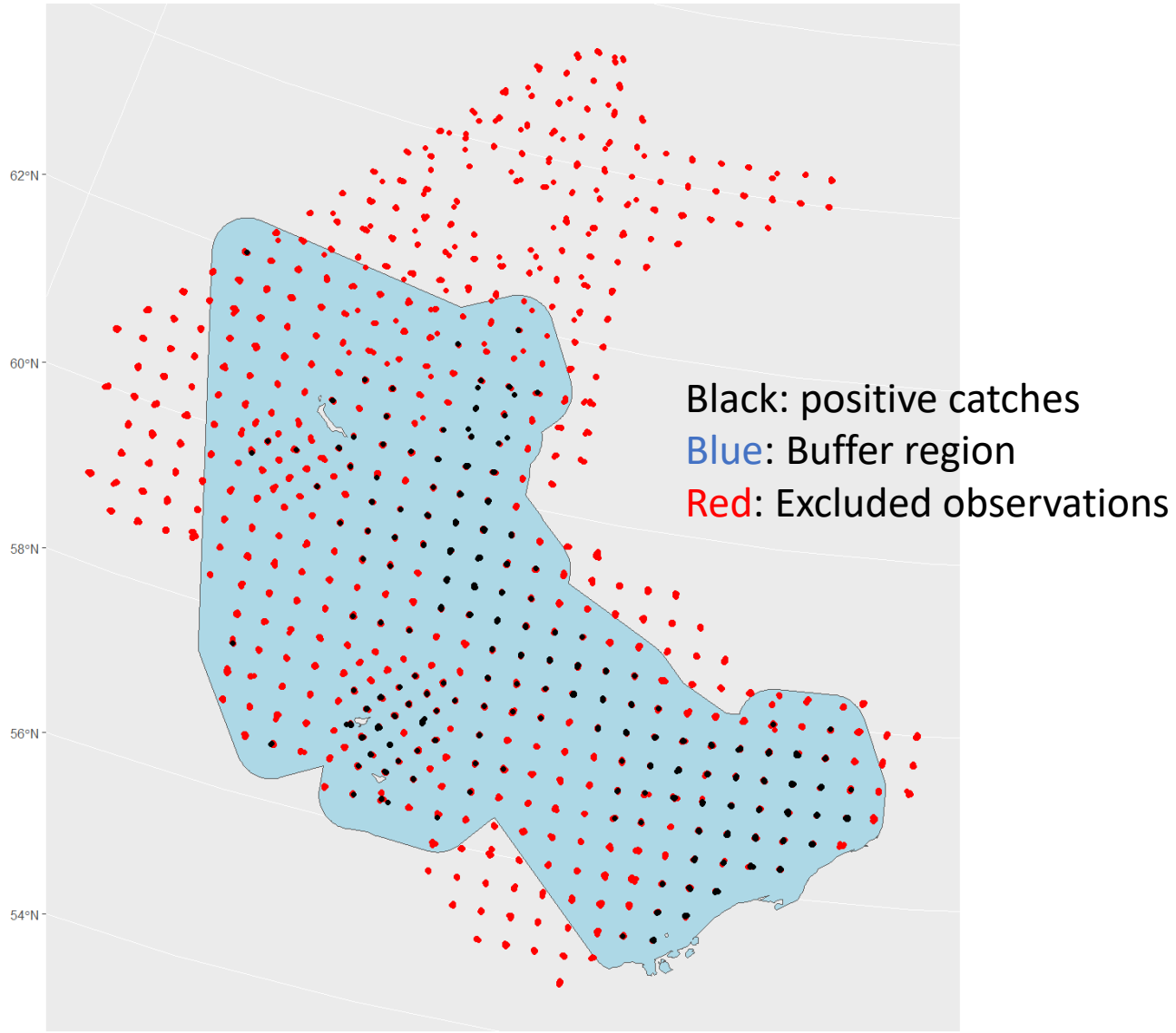
Identify the region of historical and possible occurrence

Hair crab do not occur throughout the entire range of the EBS/NBS surveys

A simple concave hull around positive catches across all years with 45 km buffer

Observations outside this region treated as “false zeros” and excluded

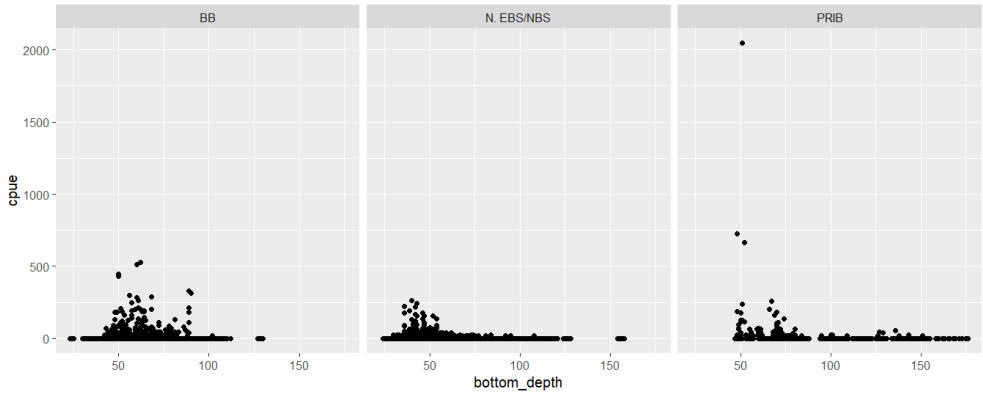
We fit the model to *all* observations within this zone, including NBS and EBS data



A few notes on the data

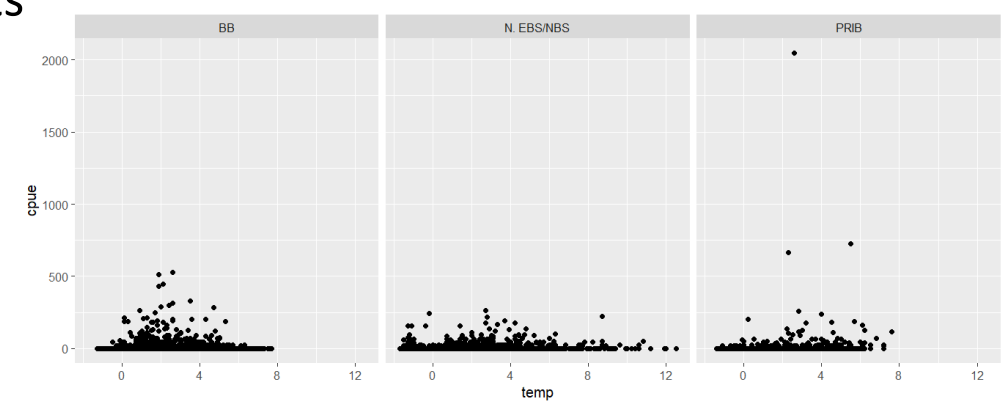
Also included in the exported data are columns for

- **gear temp** (`hair_cpue_sf_filt$temp`)
- **bottom depth** (`hair_cpue_sf_filt$bottom_depth`)



Bottom depth during trawling

Lastly, to simplify analyses, we combine “ALL” and “NBS” districts into a single variable - "N. EBS/NBS"



Gear temperature during trawling

Guidance for mesh creation

sdmTMB::make_mesh() will make a mesh for you, but is it “good”? How would we know?

To explore mesh parameterization, we will build a custom mesh using `INLA`

Steps to building a custom mesh:

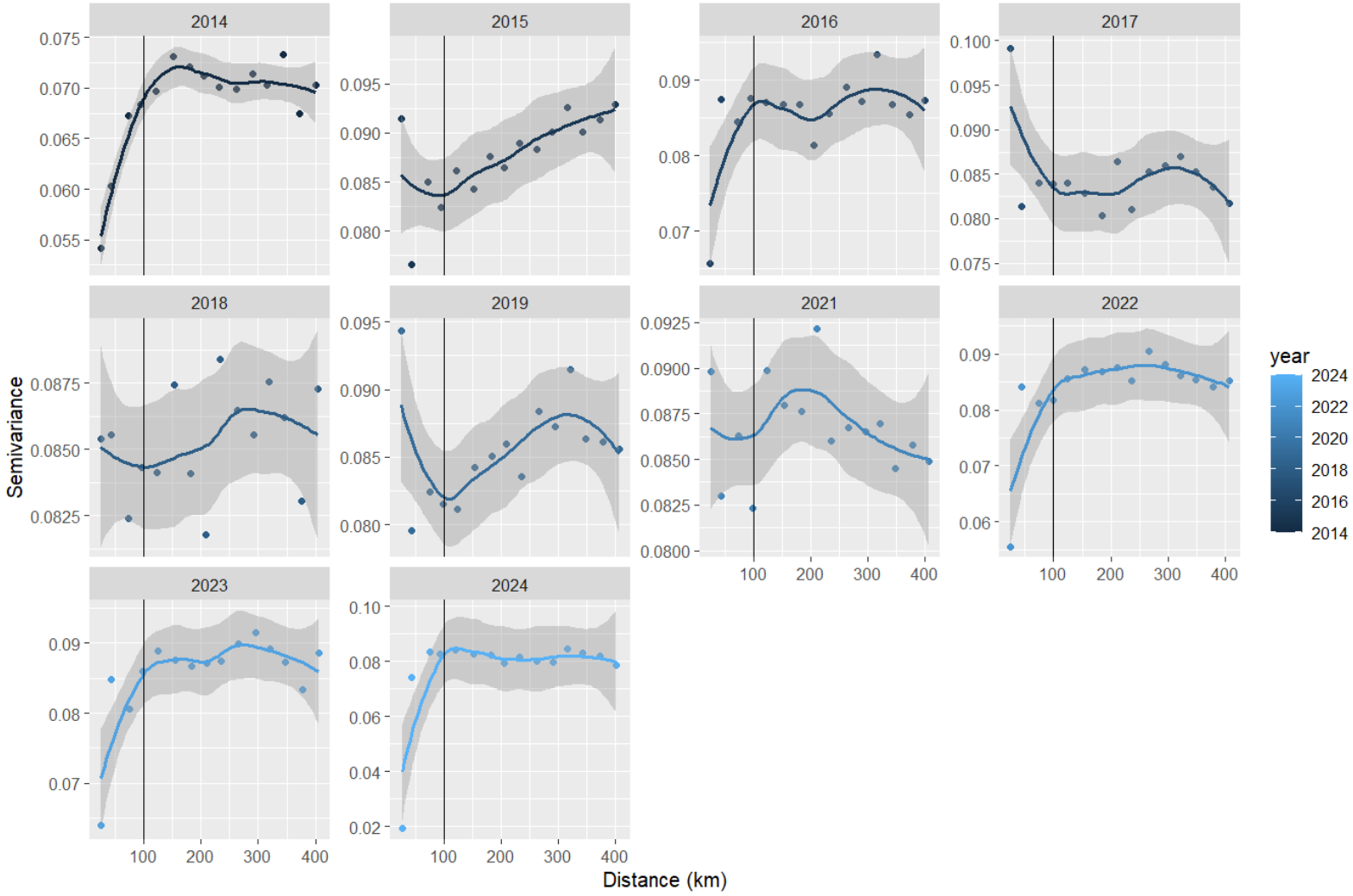
1. **Make an informed guess about the spatial range of residual spatial autocorrelation**

   - The spatial range must be estimated by the spatial GLMM, but we can make a guess to build an initial mesh that we can iterate on once the model is fitted

   - We fit a GLM with fixed-effects intercepts for “year” and use DHARMa residuals to build residual variograms by year:

```
# fit a Tweedie GLM (ignore spatial autocorrelation)
tw_mod <- sdmTMB(cpue ~ 0 + as.factor(year),
                 data = hair_cpue,
                 spatial = "off",
                 family = tweedie())
```

Guidance for mesh creation



Residual variograms reveal deviations from assumptions about how correlations decay with distance

- **Isotropy** – spatial correlation is directionally independent
- If isotropic, then semivariance should increase and plateau

Probably explained by the patchy distributional patterns of hair crab in the EBS

Mental note for later

For now, we pick a **guesstimate spatial range of 100 km** to build the mesh

Guidance for mesh creation

2. Specify the spatial domain using the `loc.domain` argument in `INLA::inla.mesh.2d`

- Creates a coarse concave hull around the observations

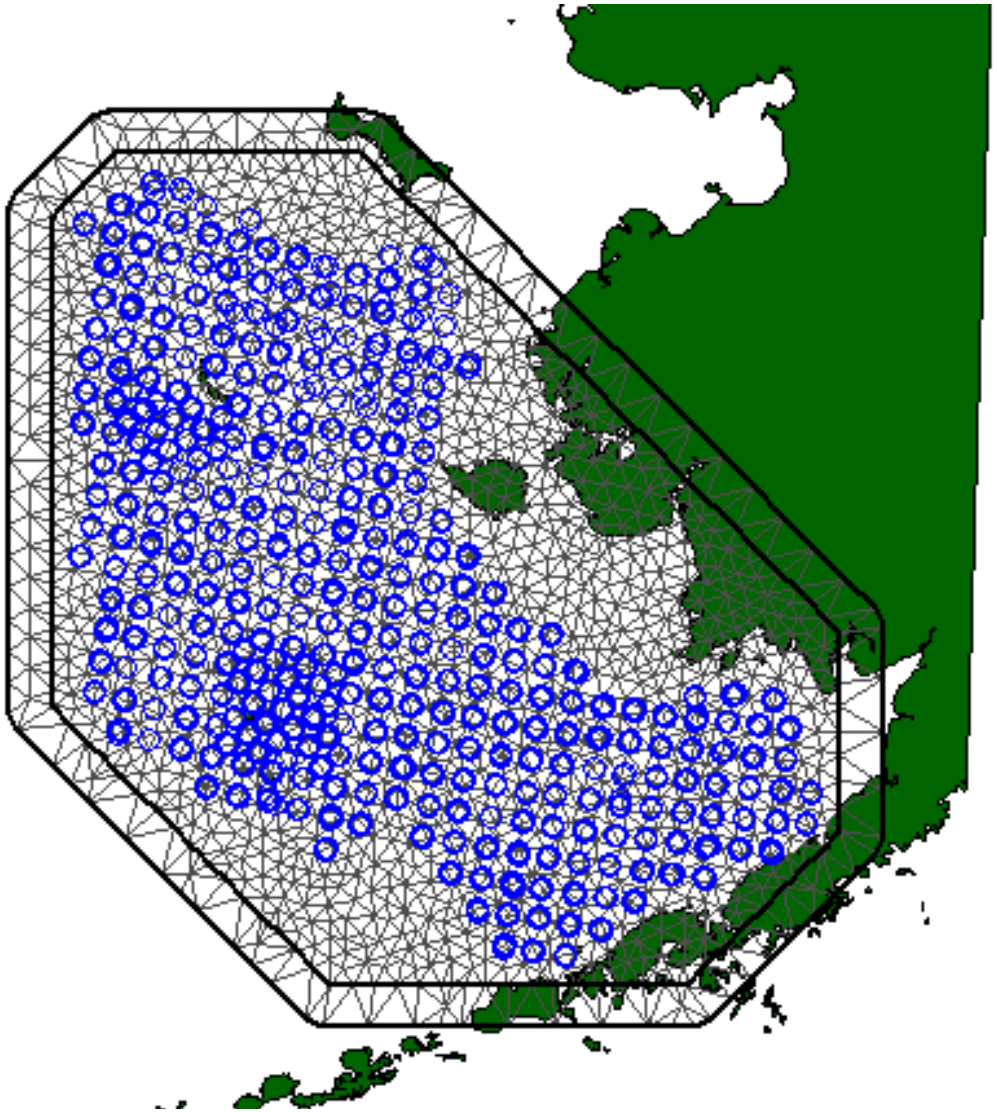
3. Specify a) mesh resolution with `max.edge` and b) the extent of the outer mesh domain using `offset`

`max.edge` – advice is between 1/10 – 1/5 of the estimated spatial range

- Can be length 2 vector: 1) inner-mesh max edge 2) outer mesh max edge length

`offset` – extent of outer boundary.

- Purpose of outer region is to minimize edge effects when estimating random fields near boundary



Guidance for mesh creation

"No weird-looking parts" – Haakon Bakka, INLA developer

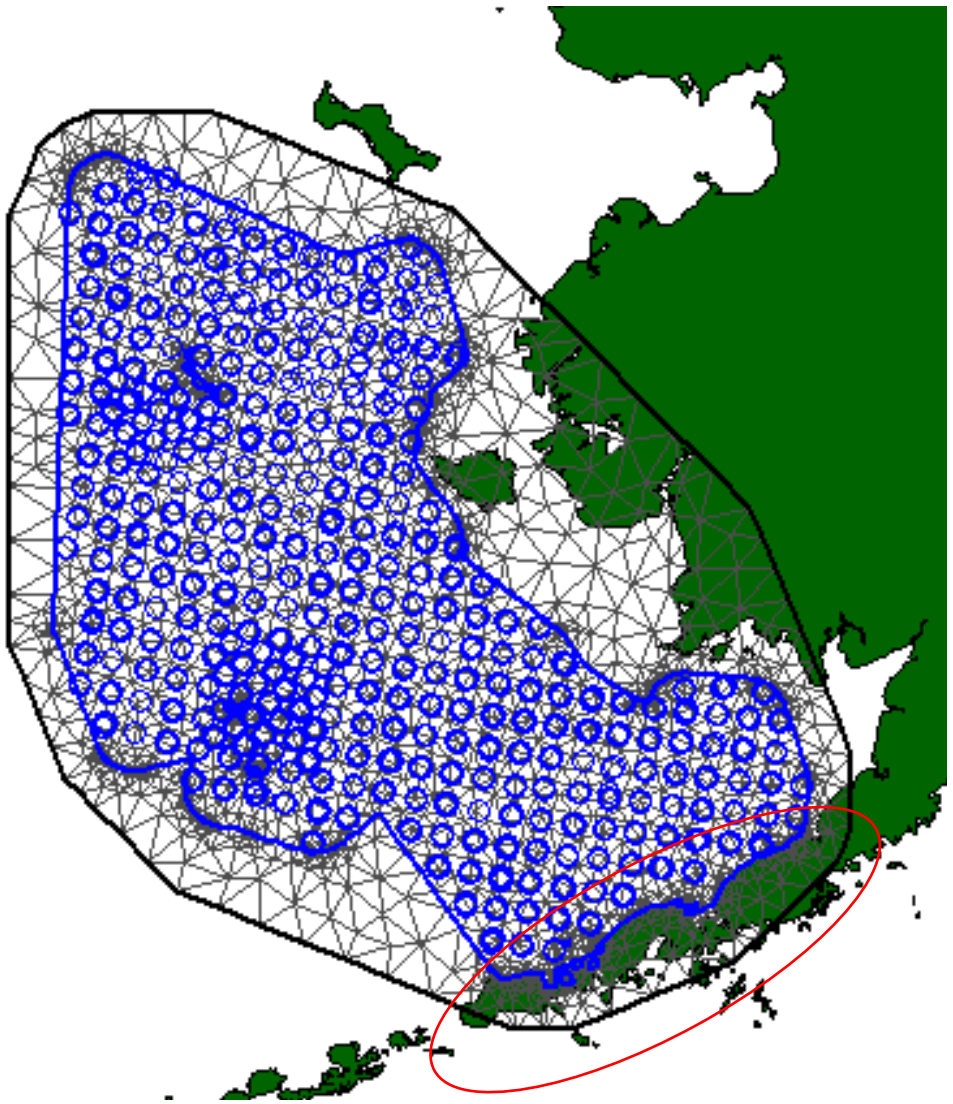
A little odd that the mesh extends over land and outside the domain of the observations

- Problem? Probably not, so long as you don't predict in these regions

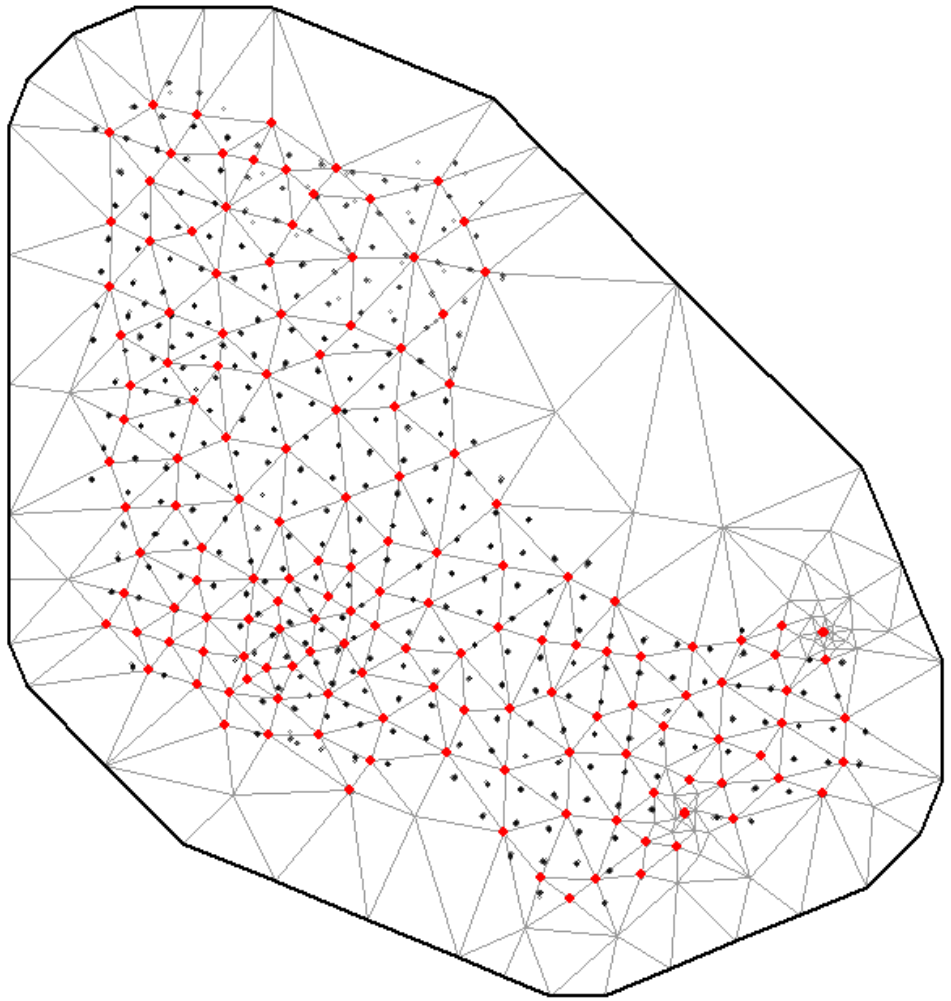
- When would it matter?

We can make the mesh more realistic by enforcing boundaries on spatial correlation

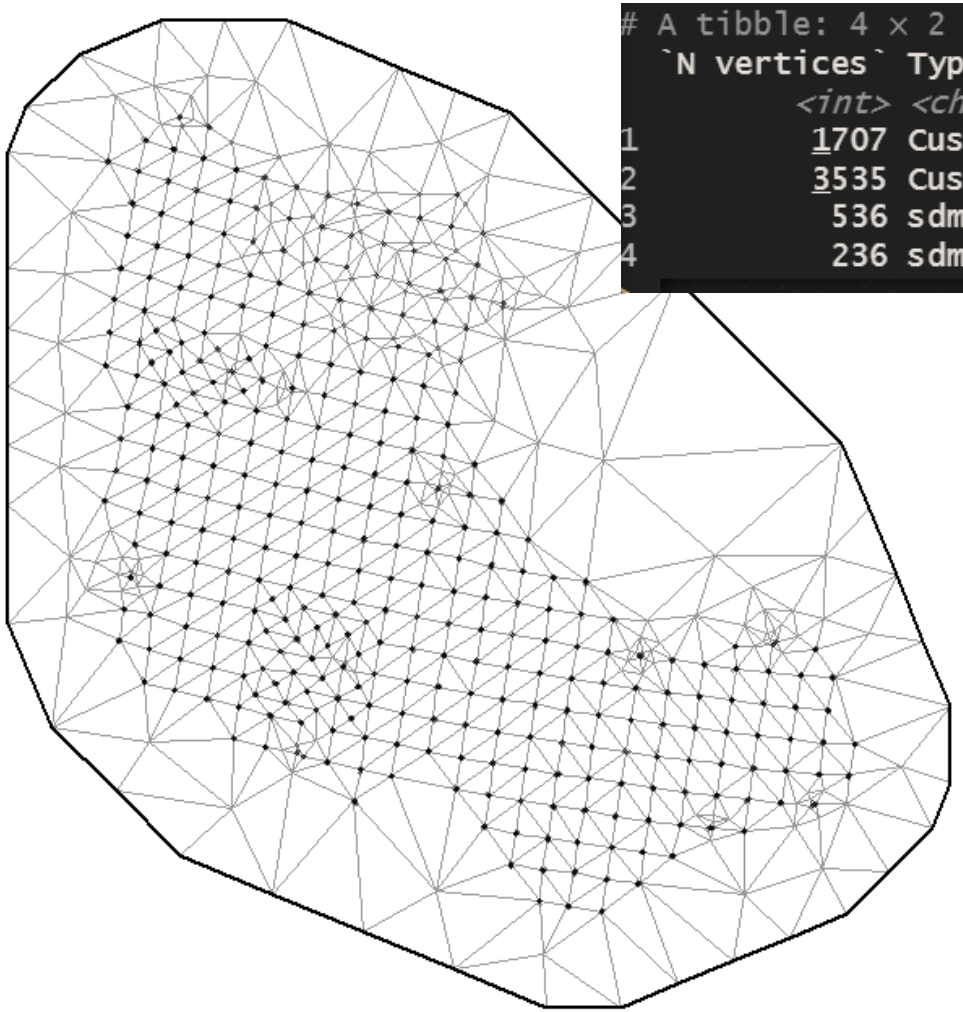
- Instead of `loc.domain`, pass sp-class polygon to `boundary` argument



Guidance for mesh creation



type = “kmeans”



```
# A tibble: 4 × 2
  `N vertices` Type
         <int> <chr>
1         1707 Custom - no barrier
2         3535 Custom - barrier
3          536 sdmTMB - cutoff
4          236 sdmTMB - kmeans
```

type = “cutoff”

Fitting the spatiotemporal model

An index standardization model seeks to develop relative or absolute indices of abundance or biomass for a stock/population over time

We can fit such a model by estimating fixed, categorical year effects while suppressing the global model intercept

```
m1 <- sdmTMB::sdmTMB(
        formula = cpue ~ 0 + as.factor(year),
        spatial = "on",
        spatiotemporal = "iid",
        time = "year",
        mesh = sdmTMB_mesh2, # simple mesh for speed
        family = tweedie(link = "log"),
        data = hair_cpue)
```

The model also estimates a spatial random field and temporally independent spatiotemporal random fields.

- When modeling spatiotemporal random fields a numeric time variable must be specified (name of column as string)

Model diagnostics

We diagnose the model for estimation problems using `sanity()`

```
> sanity(m1) #Tweedie model

✓ Non-linear minimizer suggests successful convergence
✓ Hessian matrix is positive definite
✓ No extreme or very small eigenvalues detected
✗ `ln_kappa` gradient > 0.001
i See ?run_extra_optimization(), standardize covariates, and/or simplify the model


✗ `ln_phi` gradient > 0.001
i See ?run_extra_optimization(), standardize covariates, and/or simplify the model


✓ No fixed-effect standard errors are NA
✓ No standard errors look unreasonably large
✓ No sigma parameters are < 0.01
✓ No sigma parameters are > 100
✓ Range parameter doesn't look unreasonably large
```

`ln_kappa gradient > 0.001` – Failure to estimate κ for Matérn covariance

`ln_phi gradient > 0.001` – Failure to estimate Tweedie dispersion parameter

Failure of spatial parameter estimation – maybe related to violation of isotropy assumption, among other things like an overly complex model.

- Relaxing the isotropy assumption may help the model estimate these parameters. Let's try it.

Model diagnostics

We re-fit the model and allow for *anisotropy* in spatial correlations i.e., directional dependence in spatial correlations

- `anisotropy = TRUE`

```r
m1.2 <- sdmTMB::sdmTMB(
        formula = cpue ~ 0 + as.factor(year),
        spatial = "on",
        spatiotemporal = "iid",
        time = "year",
        anisotropy = TRUE,
        mesh = sdmTMB_mesh2,  # simple mesh for speed
        family = tweedie(link = "log"),
        data = hair_cpue)
```

The updated model converges without `sanity()` problems

```
> m1.2
Spatiotemporal model fit by ML ['sdmTMB']
Formula: cpue ~ 0 + as.factor(year)
Mesh: sdmTMB_mesh2 (anisotropic covariance)
Time column: year
Data: hair_cpue
Family: tweedie(link = 'log')

Conditional model:
                    coef.est coef.se
as.factor(year)2014    -0.18    0.58
as.factor(year)2015    -0.82    0.59
as.factor(year)2016    -1.62    0.62
as.factor(year)2017    -1.12    0.59
as.factor(year)2018    -1.29    0.59
as.factor(year)2019    -1.87    0.61
as.factor(year)2021    -2.06    0.62
as.factor(year)2022    -2.03    0.62
as.factor(year)2023    -1.89    0.62
as.factor(year)2024    -1.38    0.61

Dispersion parameter: 21.70
Tweedie p: 1.01
Matérn anisotropic range (spatial): 46.4 to 208.9 at 116 deg.
Spatial SD: 2.84
Spatiotemporal IID SD: 1.24
ML criterion at convergence: 1901.838

See ?tidy.sdmTMB to extract these values as a data frame.
See ?plot_anisotropy to plot the anisotropic range.
```

The nature of `sanity()` failures can be informative for how to fix – not always necessary to “simplify”

We are returned with two spatial ranges and the dominant direction of spatial correlation

Model diagnostics

We re-fit the model and allow for *anisotropy* in spatial correlations i.e., directional dependence in spatial correlations

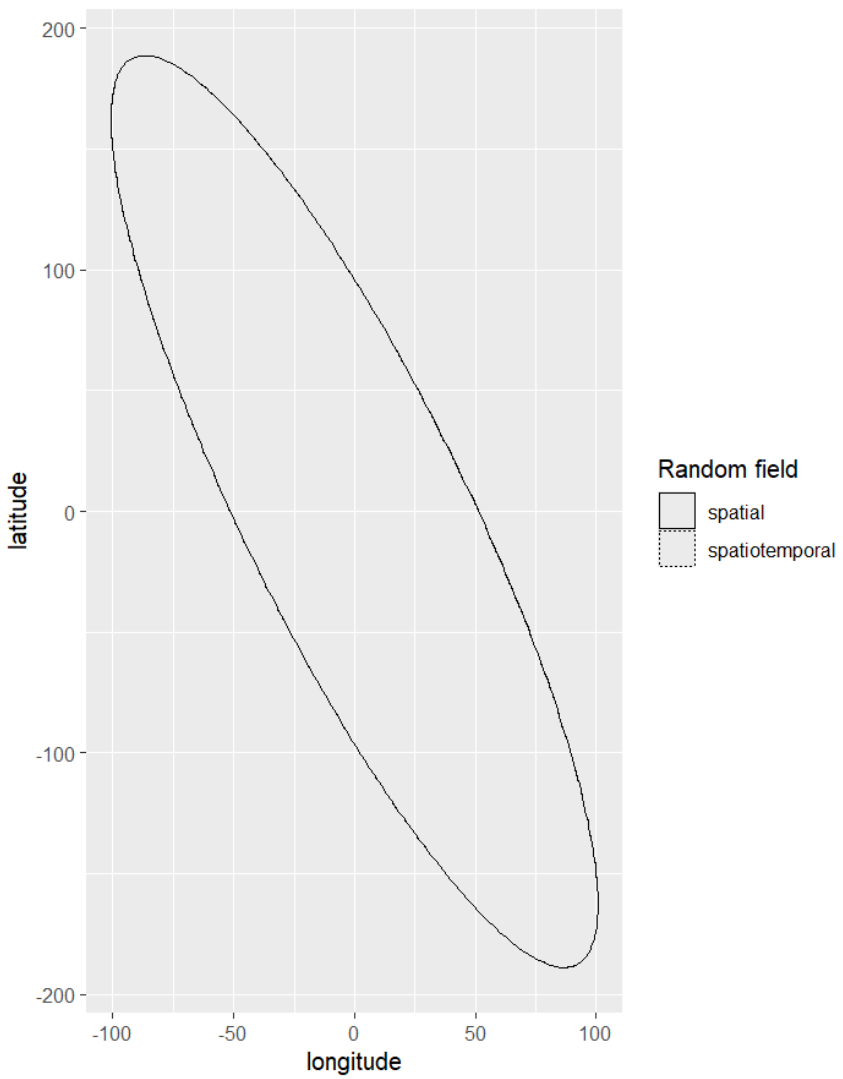
- `anisotropy = TRUE`

The updated model converges without `sanity()` problems

The nature of `sanity()` failures can be informative for how to fix – not always necessary to "simplify"

We are returned with two spatial ranges and the dominant direction of spatial correlation

- `plot_anisotropy()`
- Smaller range is the range along the minor axis, larger range for major axis of anisotropy ellipse



Evaluating residuals

Randomized quantile residuals: `residuals(…, type = “mle-mvn”)`

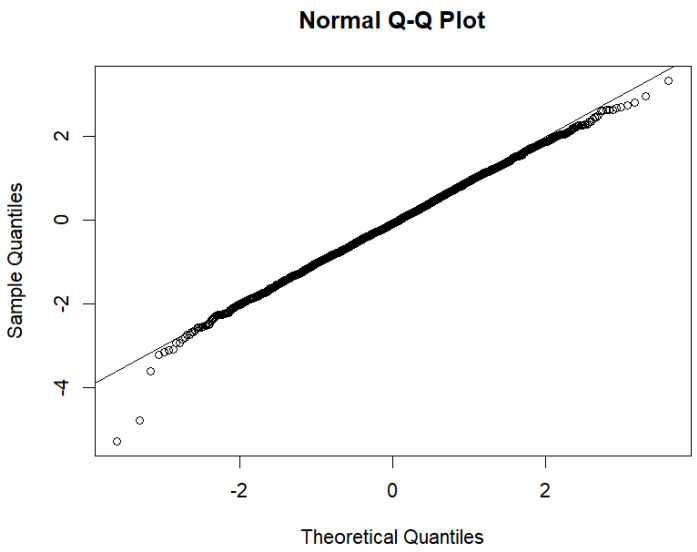
1. Residuals are randomized when integer valued
2. Transformed from response distribution to uniform for quantile residuals (as in DHARMa)
3. Transformed again from uniform to normal
   - Well-behaved residuals will be consistent with normal(0, 1)

Simulation-based quantile residuals (DHARMa – also `type = “mle-mvn”`)

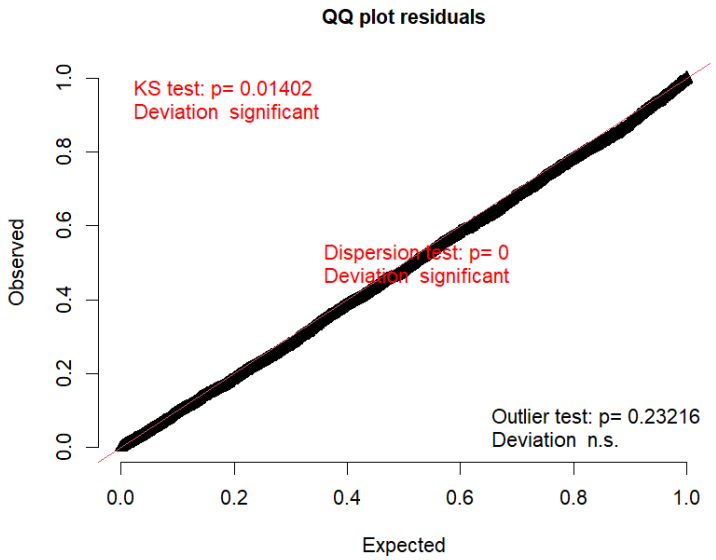
1. Simulate (rather than randomize) residuals
2. Transformed from response distribution to uniform for quantile residuals
   - Well-behaved residuals will be consistent with uniform(0, 1)

Both approaches calculate residuals as if uncertainty in random effects [conditional on data] is well-described by multivariate normal distribution (“…-mvn”)

- Residuals calculated by sampling **once** from this MVN conditional on MLE values for fixed effects parameters
- QQ-plots change with each run – set a seed and avoid fishing



Quantile residual qq-plot for normal(0,1)



plot(DHARMa) yields a qq-plot for uniform(0,1)

Evaluating residuals

MCMC-based randomized quantile residuals

- Relaxes the assumption that random effects can be approximated by a MVN
- Samples the random effects from the posterior conditional on fixed effects held at their MLEs

Time-intensive to generate and iterate upon but robust – “an excellent option” according to vignette

May fail with default `mcmc_iter` and `mcmc_warmup`

- Issues may resolve with more iterations

```
Chain 1:  Elapsed Time: 546.391 seconds (Warm-up)
Chain 1:                484.222 seconds (Sampling)
Chain 1:                1030.61 seconds (Total)
Chain 1:

Warning messages:
1: The largest R-hat is 1.12, indicating chains have not mixed.
Running the chains for more iterations may help. See
https://mc-stan.org/misc/warnings.html#r-hat
2: Bulk Effective Samples Size (ESS) is too low, indicating posterior means and medians may be unreliable.
Running the chains for more iterations may help. See
https://mc-stan.org/misc/warnings.html#bulk-ess
3: Tail Effective Samples Size (ESS) is too low, indicating posterior variances and tail quantiles may be unreliable.
Running the chains for more iterations may help. See
https://mc-stan.org/misc/warnings.html#tail-ess
```

Best to experiment with several options - I have ended with MCMC residuals for publications

Generating the prediction grid

The NMFS Eastern Bering Sea bottom trawl survey is grid-based with survey stations spaced 20 nm apart

- Everything in this analysis is in km

See `data/SAP_layers.gdb` for gridded polygon of the EBS survey region used in the calculation of design-based abundance indices

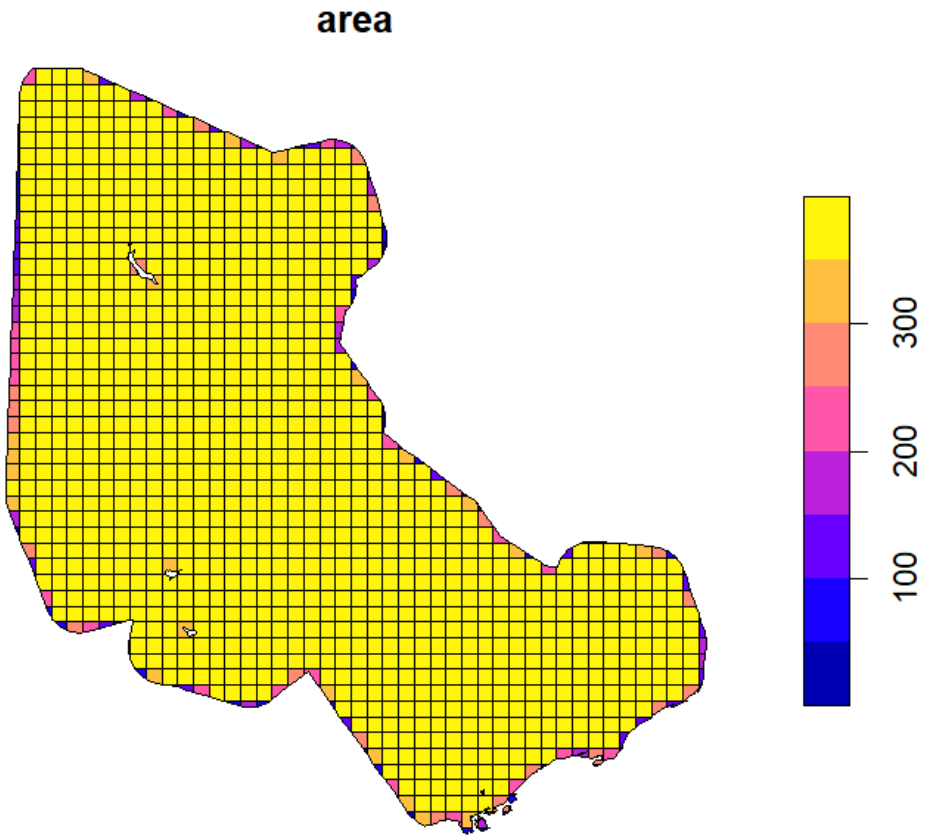
In `sdmTMB_index_standardization.qmd`

- Choose design-based EBS grid or create your own with custom spatial resolution by setting `custom_resolution = TRUE` or `FALSE`

In our case, generating a custom prediction grid is simple

- Take a polygon of EBS and turn it into a grid
- Intersect that polygon with the buffer polygon used to filter observations – now a *prediction grid* specific to EBS*
- Expand the prediction grid across all time steps
- Extract grid cell centroids for prediction
- Convert to `data.frame` with lon/lat columns for use in `predict(…)`

Gets more involved when covariates are included – covariate observations need to be available for all time steps and locations



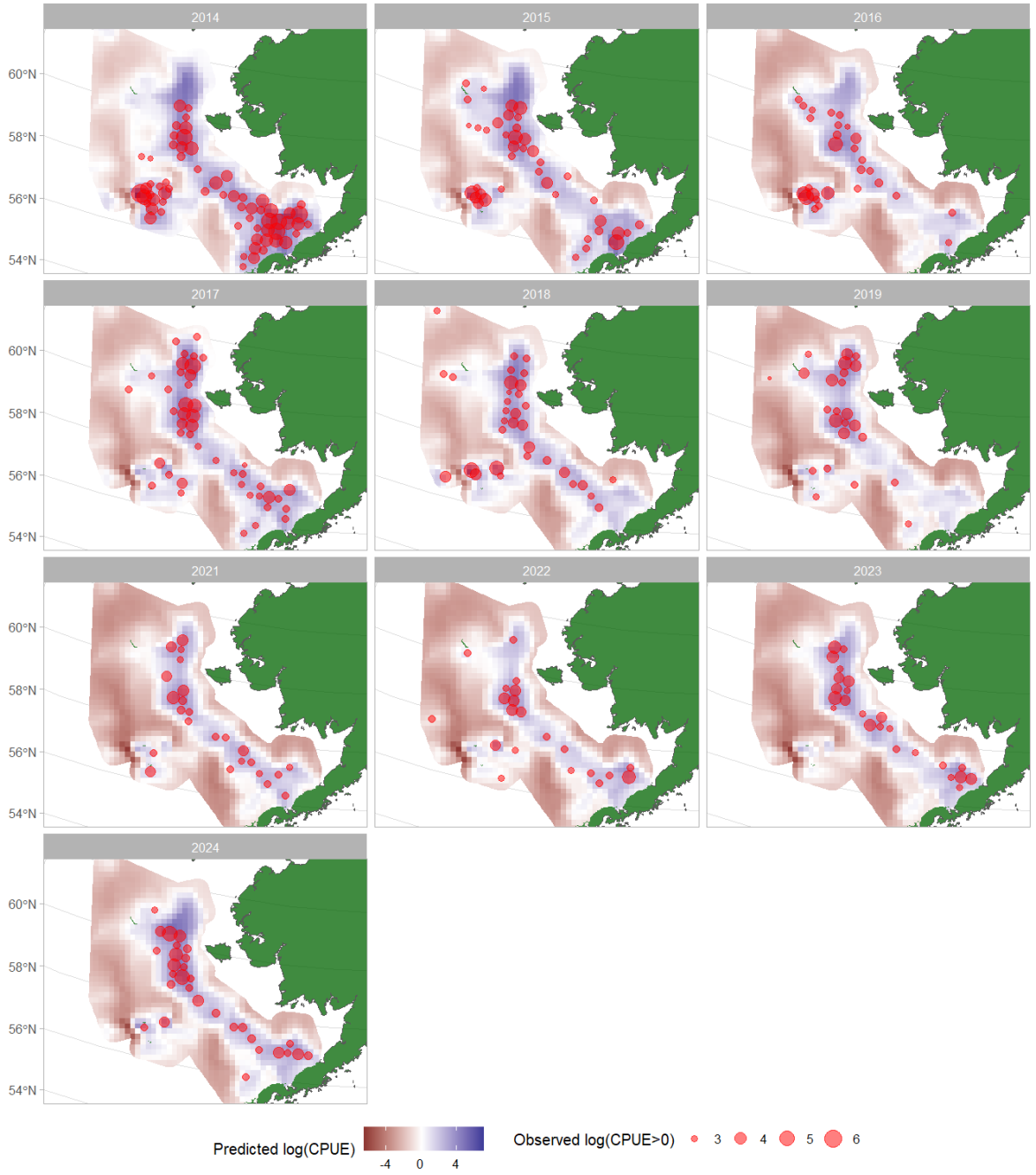
Predicting from the model

```
> model_predictions_df <- predict(m1.2, newdata = pred_df, type = "link",
+                                 return_tmb_object = TRUE)
> model_predictions_df$data
# A tibble: 12,340 × 10
      id   area  year longitude latitude    est est_non_rf est_rf omega_s epsilon_st
   <int>  <dbl> <int>     <dbl>    <dbl>  <dbl>      <dbl>  <dbl>   <dbl>      <dbl>
 1     1 0.0540  2014     -605.     596.  0.886     -0.178   1.06   0.543      0.521
 2     1 0.0540  2015     -605.     596. -0.230     -0.822  0.591   0.543     0.0485
 3     1 0.0540  2016     -605.     596. -1.22      -1.62   0.398   0.543    -0.145
 4     1 0.0540  2017     -605.     596. -0.453     -1.12   0.662   0.543      0.119
 5     1 0.0540  2018     -605.     596. -0.927     -1.29   0.358   0.543    -0.185
 6     1 0.0540  2019     -605.     596. -1.28      -1.87   0.595   0.543     0.0520
 7     1 0.0540  2021     -605.     596. -1.60      -2.06   0.464   0.543    -0.0794
 8     1 0.0540  2022     -605.     596. -1.57      -2.03   0.456   0.543    -0.0869
 9     1 0.0540  2023     -605.     596. -1.47      -1.89   0.426   0.543    -0.117
10     1 0.0540  2024     -605.     596. -0.865     -1.38   0.520   0.543    -0.0235
```

Predictions returned on link scale by default

est = predicted log CPUE
est_non_rf = log CPUE associated with year-intercepts (varies by year)
omega_s = log CPUE associated with spatial random field (static)
epsilon_st = log CPUE associated with spatiotemporal random field (varies by year)
est_rf = omega_s + epsilon_st

For calculating abundance indices, `return_tmb_object = TRUE`



Calculating an abundance index: two ways

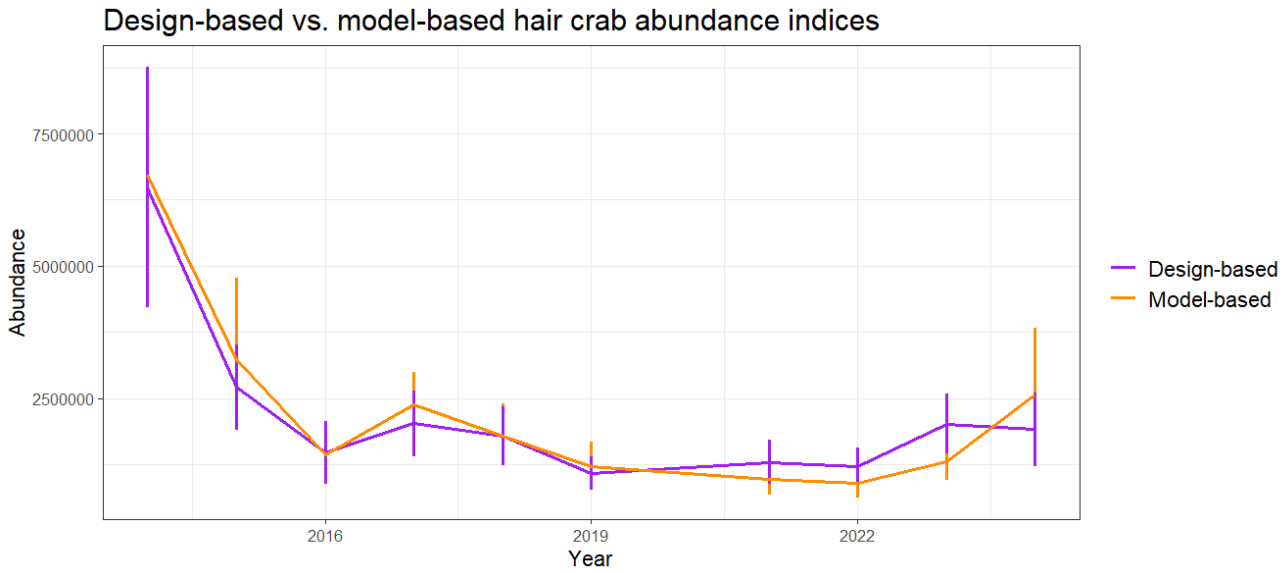
Multiple ways to calculate abundance indices depending on need

- We are working on catch per unit area swept, so summing index predictions over space by year is reasonable choice (the default)

- We pass a vector of grid cell areas to `get_index`

- `get_index` calculates standard errors on predictions using the *delta method* in TMB

```
> model_based_index <-
+   get_index(obj = model_predictions_df,
+             bias_correct = TRUE,
+             area = pred_df$area)
```



What about something like catch per pot or numbers observed per hour?

- May make more sense to generate an index that remains in relative units

Calculating an abundance index: simulated-based indices

Set `nsim = …` in `predict()` to simulate predictions from the joint precision matrix of the fitted model

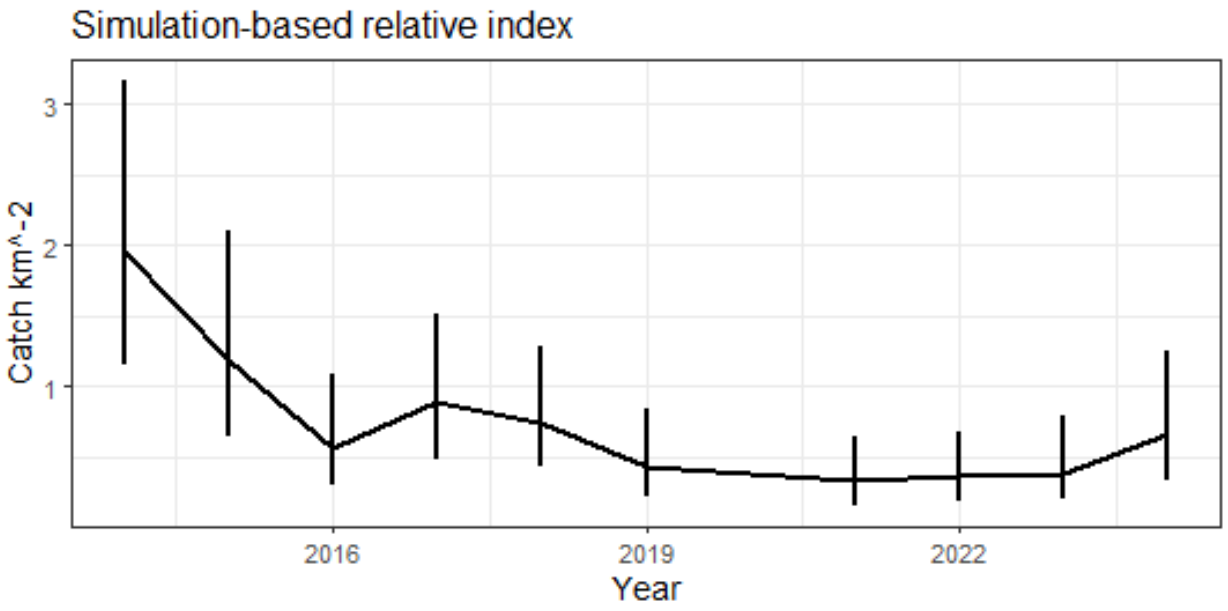
Pass output to `get_index_sims()` with at least two arguments:

1. `agg_function`: the function that will aggregate across `nsim` simulations within each time step

2. `est_function`: the function that will aggregate across the distribution of indices produced by `agg_function`

We can ignore area argument

```
> model_predictions_df_sim <- predict(m1.2,
+                                     newdata = pred_df,
+                                     nsim = 1000,
+                                     type = "link")
>
> model_based_index_relative <-
+   get_index_sims(obj = model_predictions_df_sim,
+                  agg_function = function(x) median(exp(x)),
+                  est_function = median)
```



Model comparison and selection

Please turn your scripts to `R/sdmTMB_model_comparison.qmd`

We fit the following models that differ in

1. The allowed basis complexity of a smooth effect of depth on CPUE
2. The inclusion of a categorical "district" term

```
m2 <- sdmTMB::sdmTMB(
        formula = cpue ~ (1|yf) + s(bottom_depth, m = 1, k = 5),
        spatial = "on",
        anisotropy = TRUE,
        mesh = sdmTMB_mesh2,
        family = tweedie(link = "log"),
    data = hair_cpue)
```

Models pass `sanity()` and have similarly behaved residuals

```
m3 <- sdmTMB::sdmTMB(
        formula = cpue ~ (1|yf) + s(bottom_depth, m = 1) + district,
        spatial = "on",
        anisotropy = TRUE,
        mesh = sdmTMB_mesh2,
        family = tweedie(link = "log"),
    data = hair_cpue)
```

Model comparison and selection

Please turn your scripts to `R/sdmTMB_model_comparison.qmd`

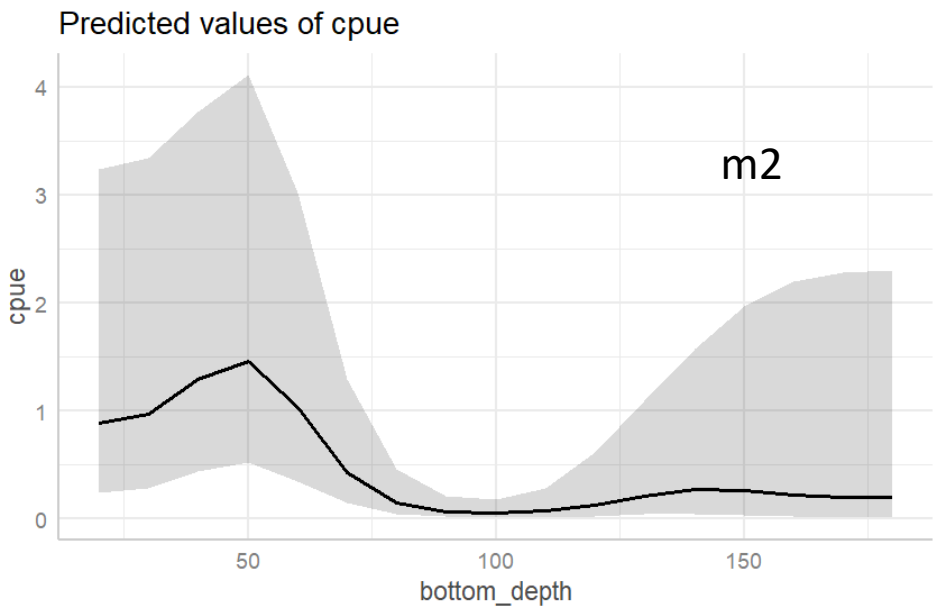
We fit the following models that differ in

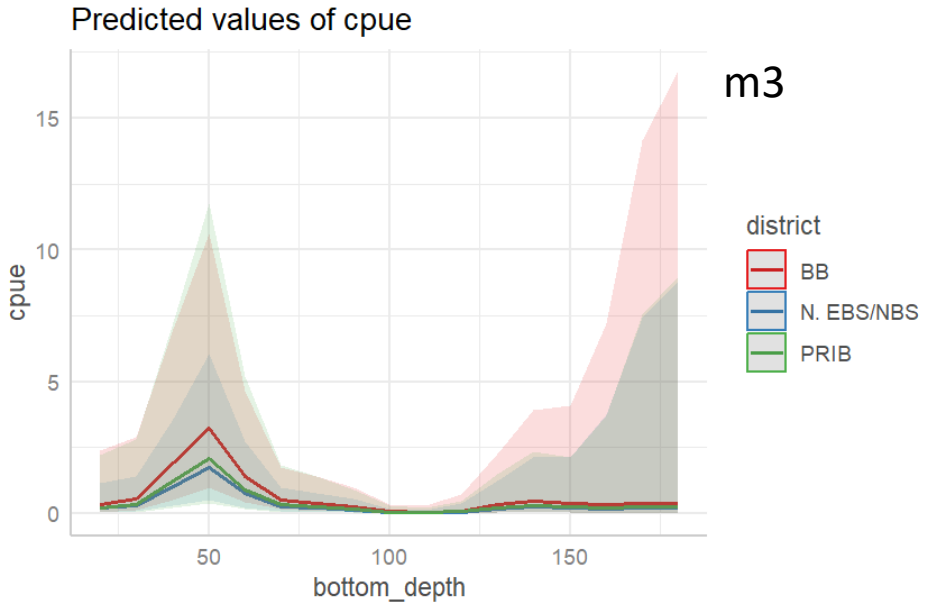
1. The allowed basis complexity of a smooth effect of depth on CPUE
   - Smooths estimated as random effects via `mgcv::smooth2random` – no FE parameters
2. The inclusion of a categorical "district" term

Models pass `sanity()` and have similarly behaved residuals

Visualizing the marginal smooth effects using `ggeffects::ggpredict()`

Which is the better model?





Model comparison and selection: AIC/cAIC

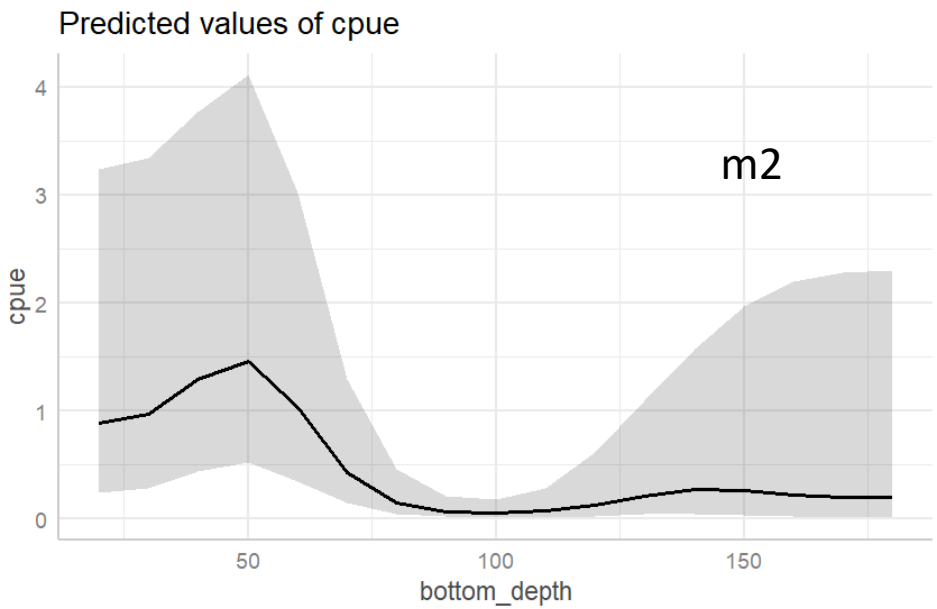
Quick marginal AIC (AIC) and conditional AIC (cAIC) checks

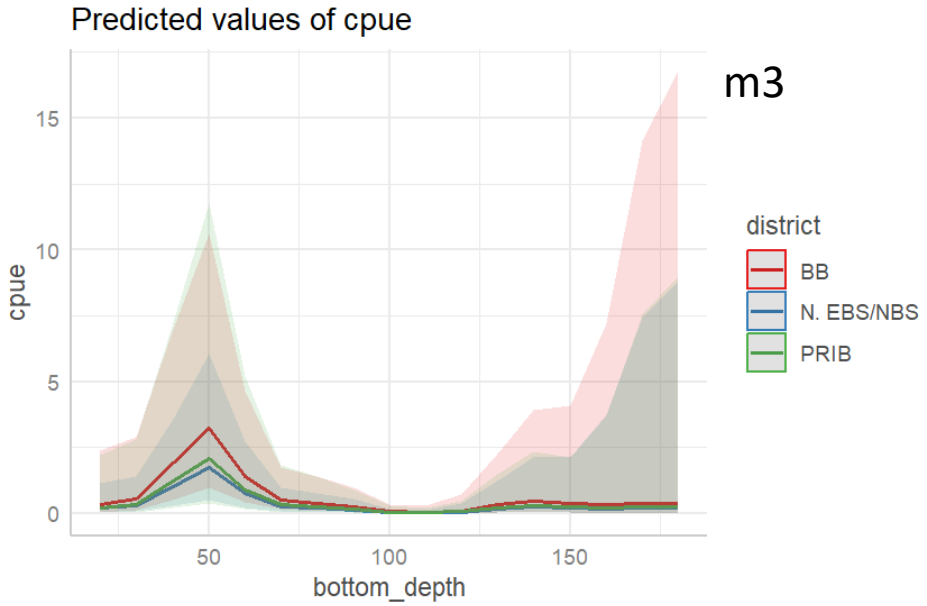
For spatiotemporal models:

- AIC – Optimizes for **extrapolation** predictive performance (predicting outside of spatial/spatiotemporal domain)
- cAIC – Optimizes for **interpolation** predictive performance (predicting on unmeasured locations within spatial/spatiotemporal domain)

Both AIC and cAIC support `m3` as the better model

We can also apply *k*-fold cross validation to compare prediction skill





Model comparison and selection: cross validation

sdmTMB_cv applies *k*-fold cross validation by default:

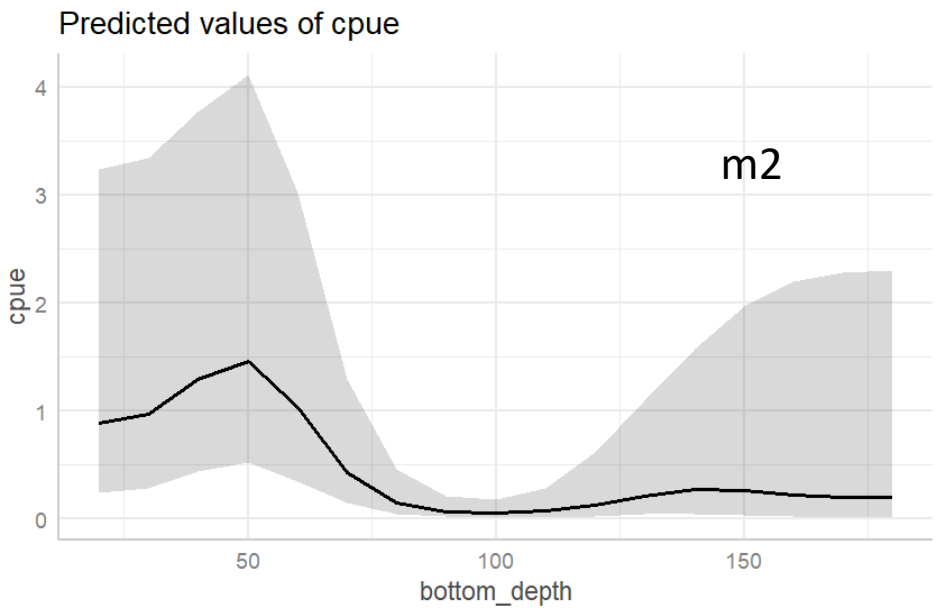
- The data are broken into *k* "folds"
- One fold is held out, the model is fitted
- The model predicts on the held out data and evaluates prediction skill

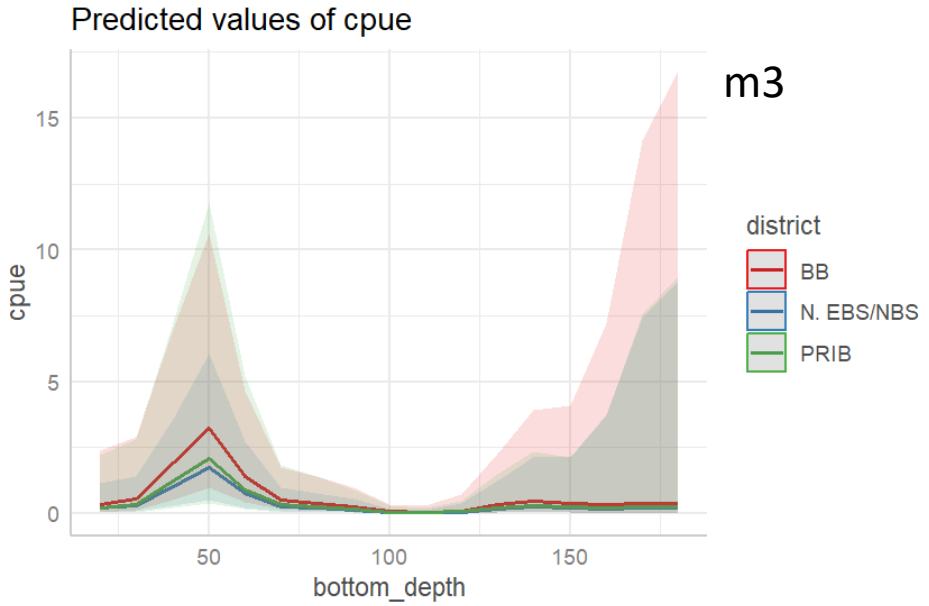
Key measure of prediction skill is the sum of the log-likelihoods across held out data – "`sum_loglik`"

- More positive values are better

I like more interpretable measures like mean absolute error (MAE) or root mean squared error (RMSE) across held out data

- MAE and RMSE are virtually equivalent for these models – little to be gained from the added complexity in `m3`





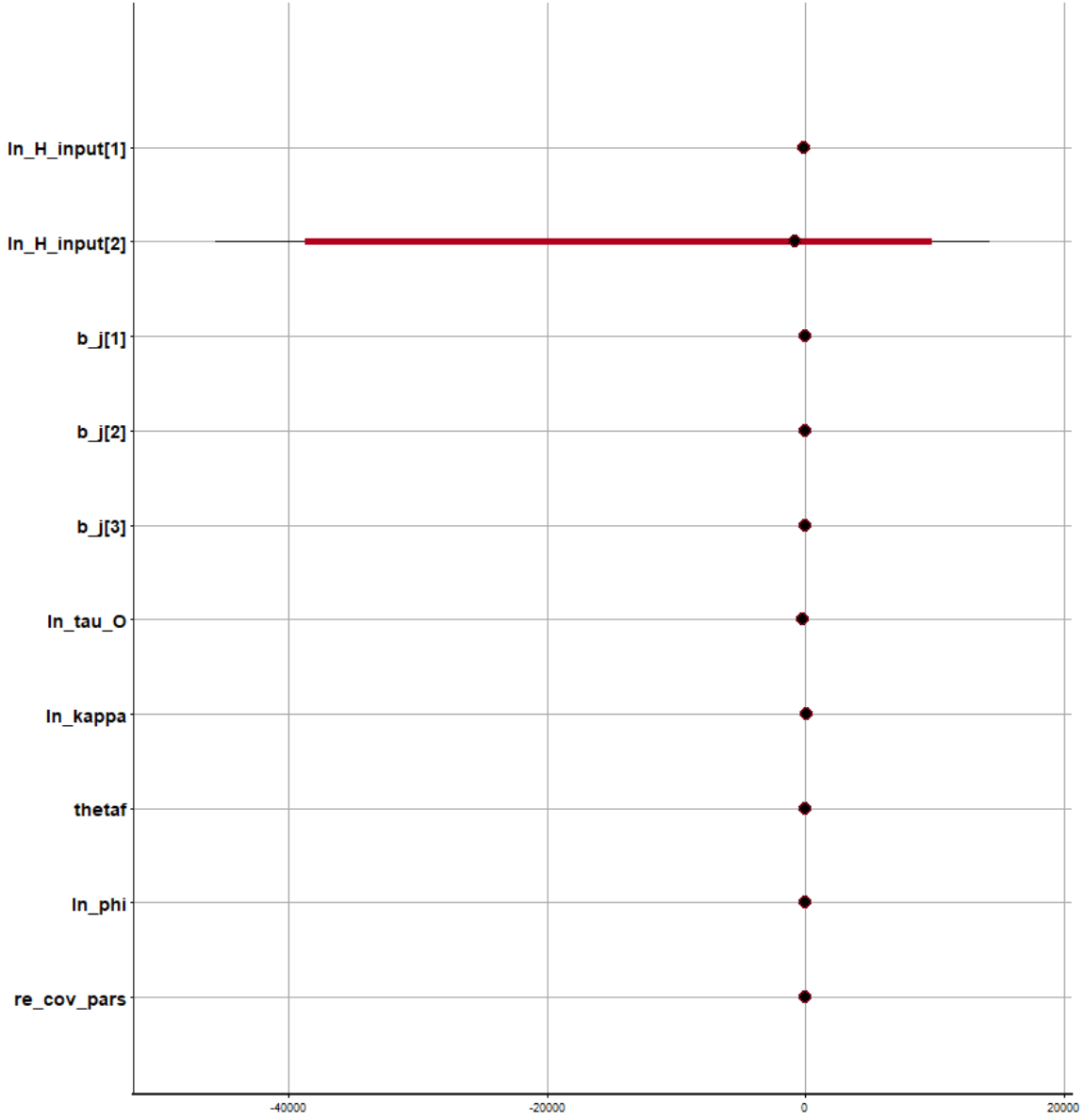
Bayesian inference with tmbstan

We can do Bayesian inference on parameters by passing `sdmTMB` models fitted with `bayesian = TRUE` to `tmbstan`

The Bayesian version of m3 was initially not well estimated by tmbstan

Huge standard errors on the anisotropy term

What if we re-fit without anisotropy?



Bayesian inference with tmbstan

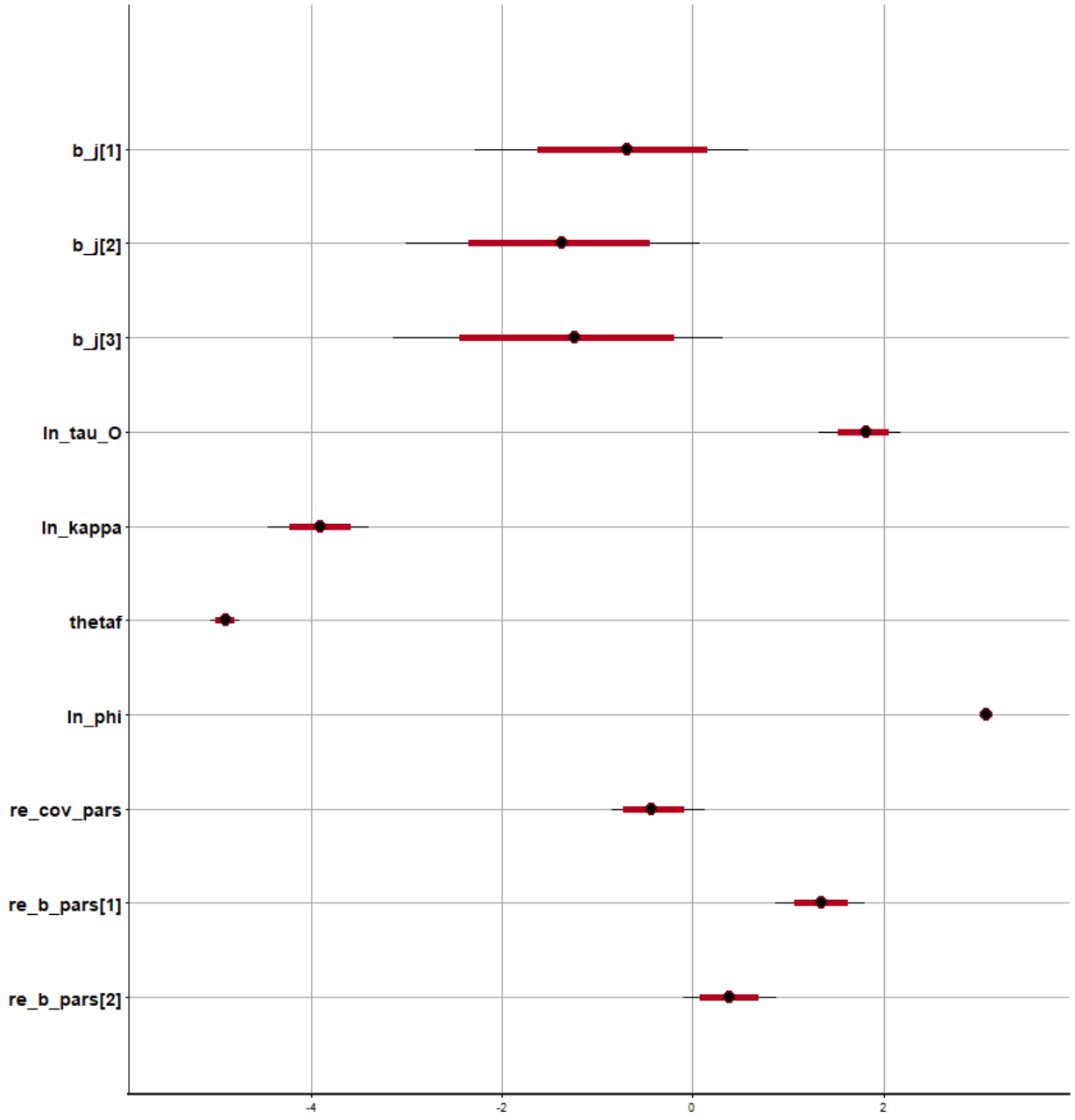
Removing the anisotropy term and iterating on tmbstan defaults improved MCMC performance

- Still getting errors on R-hat – may improve if more chains or sampling iterations

plot(fit_stan) to visualize first 10 parameter posterior distributions

b_j terms are varying intercepts for `district`

- Suggests that hair crab CPUE is similar across districts



Spatiotemporal modeling lab

Please turn your scripts to `R/sdmTMB_lab_exercises.qmd`